



LJ University
University with a Difference

DATA STRUCTURE USING JAVA

LECTURE NOTES FOR UNIT - 3

UNIT -3 STACK - II

PREPARED BY:
MR. VISMAY SHAH

Table of Contents

3.1	Chapter Introduction	2
3.3	Definition of Infix, Postfix, and Prefix	4
3.4	How To Check If The Given Expression Is Valid Or Not?	5
3.5	Operator Precedence And Associativity	6
3.6	Expression Evaluation & Conversion.....	8
3.7	Infix to Postfix Conversion	9
a.	Infix to Postfix Conversion Without Parenthesis	10
b.	ALGORITHM - Infix To Postfix Conversion Without Parenthesis.....	11
c.	EXAMPLES – Infix Expression to Postfix Conversion Without Parenthesis	12
d.	Infix to Postfix Conversion With Parenthesis.....	18
e.	ALGORITHM - Infix To Postfix Conversion With Parenthesis	19
3.8	Postfix to Infix Conversion Using Stack	24
a.	EXAMPLES – Postfix Expression to Infix Conversion Using Stacks	25
3.9	Infix to Prefix Conversion	28
a.	Examples - Infix to Prefix Conversion Without Parenthesis.....	29
b.	Examples - Infix to Prefix Conversion With Parenthesis	32
3.10	Prefix to Infix Conversion Using Stack.....	36
a.	Examples – Prefix to Infix Conversion Using Stack	37
3.11	Prefix to Postfix Conversion Using Stack.....	40
a.	Examples - Prefix to Postfix Conversion Using Stack	41

3.1 Chapter Introduction

When you write an arithmetic expression such as $B * C$, the form of the expression provides you with information so that you can interpret it correctly. In this case we know that the variable B is being multiplied by the variable C since the multiplication operator $*$ appears between them in the expression. This type of notation is referred to as infix since the operator is in between the two operands that it is working on.

Consider another infix example, $A + B * C$. The operators $+$ and $*$ still appear between the operands, but there is a problem. Which operands do they work on? Does the $+$ work on A and B or does the $*$ take B and C? The expression seems ambiguous.

In fact, you have been reading and writing these types of expressions for a long time and they do not cause you any problem. The reason for this is that you know something about the operators $+$ and $*$. Each operator has a precedence level. Operators of higher precedence are used before operators of lower precedence. The only thing that can change that order is the presence of parentheses. The precedence order for arithmetic operators' places multiplication and division above addition and subtraction. If two operators of equal precedence appear, then a left-to-right ordering or associativity is used.

Let's interpret the troublesome expression $A + B * C$ using operator precedence. B and C are multiplied first, and A is then added to that result. $(A + B) * C$ would force the addition of A and B to be done first before the multiplication. In expression $A + B + C$, by precedence (via associativity), the leftmost $+$ would be done first.

Although all this may be obvious to you, remember that computers need to know exactly what operators to perform and in what order. One way to write an expression that guarantees there will be no confusion with respect to the order of operations is to create what is called a fully parenthesized expression. This type of expression uses one pair of parentheses for each operator. The parentheses dictate the order of operations; there is no ambiguity. There is also no need to remember any precedence rules.

The expression $A + B * C + D$ can be rewritten as $((A + (B * C)) + D)$ to show that the multiplication happens first, followed by the leftmost addition. $A + B + C + D$ can be written as $((((A + B) + C) + D))$ since the addition operations associate from left to right.

There are two other very important expression formats that may not seem obvious to you at first. Consider the infix expression $A + B$. What would happen if we moved the operator before the two operands? The resulting expression would be $+ A B$. Likewise, we could move the operator to the end. We would get $A B +$. These look a bit strange.

These changes to the position of the operator with respect to the operands create two new expression formats, prefix and postfix. Prefix expression notation requires that all operators precede the two operands that they work on. Postfix, on the other hand, requires that its operators come after the corresponding operands. A few more examples should help to make this a bit clearer (see Table 1).

Infix Expression	Prefix Expression	Postfix Expression
$A + B$	$+ A B$	$A B +$
$A + B * C$	$+ A * B C$	$A B C * +$

$A + B * C$ would be written as $+ A * B C$ in prefix. The multiplication operator comes immediately before the operands B and C, denoting that $*$ has precedence over $+$. The addition operator then appears before the A and the result of the multiplication.

In postfix, the expression would be $A B C * +$. Again, the order of operations is preserved since the $*$ appears immediately after the B and the C, denoting that $*$ has precedence, with $+$ coming after. Although the operators moved and now appear either before or after their respective operands, the order of the operands stayed exactly the same relative to one another.

Now consider the infix expression $(A + B) * C$. Recall that in this case, infix requires the parentheses to force the performance of the addition before the multiplication. However, when $A + B$ was written in prefix, the addition operator was simply moved before the operands, $+ A B$. The result of this operation becomes the first operand for the multiplication. The multiplication operator is moved in front of the entire expression, giving us $* + A B C$. Likewise, in postfix $A B +$ forces the addition to happen first. The multiplication can be done to that result and the remaining operand C. The proper postfix expression is then $A B + C *$.

Consider these expressions again (see Table 1 & 2). Something very important has happened. Where did the parentheses go? Why don't we need them in prefix and postfix? The answer is that the operators are no longer ambiguous with respect to the operands that they work on. Only infix notation requires the additional symbols. The order of operations within prefix and postfix expressions is completely determined by the position of the operator and nothing else. In many ways, this makes infix the least desirable notation to use.

Infix Expression	Prefix Expression	Postfix Expression
$(A + B) * C$	$* + A B C$	$A B + C *$

3.2 Application of Stack: Infix, Prefix and Postfix

- For writing complex mathematical expressions, we generally prefer parentheses to make them more readable. In computers, expressions with various parentheses add unnecessary work while solving. So, to minimize the computational works, various notations have been made. Prefix and Postfix are one of them.
- In this chapter, we'll be learning about infix, postfix, and prefix conversion in detail. You'll find questions on infix, postfix, and prefix conversion in the frequently asked interview questions of almost every top tech-based company. So, let's get started!

3.3 Definition of Infix, Postfix, and Prefix

- **Infix:** The typical mathematical form of expression that we encounter generally is known as infix notation. **In infix form, an operator is written in between two operands.**

For example:

An expression in the form of $A * (B + C) / D$ is in infix form. This expression can be simply decoded as: *"Add B and C, then multiply the result by A, and then divide it by D for the final answer."*

- **Prefix:** **In prefix expression, an operator is written before its operands. This notation is also known as "Polish notation".**

For example, The above expression can be written in the prefix form as $/ * A + B C D$. This type of expression cannot be simply decoded as infix expressions.

- **Postfix:** **In postfix expression, an operator is written after its operands. This notation is also known as "Reverse Polish notation".**

For example, The above expression can be written in the postfix form as $A B C + * D /$. This type of expression cannot be simply decoded as infix expressions.

Infix	Prefix	Postfix
$A+B$	$+AB$	$AB+$
$A+B-C$	$-+ABC$	$AB+C-$
$(A+B) * C-D$	$-*+ABCD$	$AB+C*D-$

Prefix and postfix notions are methods of writing mathematical expressions without parentheses. Let's see the infix, postfix and prefix conversion. In this entire chapter, we are going to convert and evaluate expression from Infix to Postfix, Infix to Postfix, Postfix to Infix, and so on.

3.4 How To Check If The Given Expression Is Valid Or Not?

When you are working with the expressions, before converting the expression in any other form – we must check that the expression is valid or not.

And to do so – we need to find the Rank of the Expression.

For Any Expression To Be Considered as Valid

The difference between the total operand and total operators in the expression shall be 1.

Total No. of Operands in Expression – Total No. of Operators in Expression = 1

Examples:

Infix	Prefix	Rank	Valid or Invalid
a+*b	ab*+	0	Invalid
a-b*c	abc*-	1	Valid
ab+c	abc+	2	Invalid
(a+b)*(c-d)	ab+cd-*	1	Valid
a+b/d-	abd/+-	0	Invalid

3.5 Operator Precedence And Associativity

Precedence	Operator	Description	Associativity
1	++ --	Suffix/postfix increment and decrement	Left-to-right
	()	Function call	
	[]	Array subscripting	
	.	Structure and union member access	
	->	Structure and union member access through pointer	
	(type){list}	Compound literal(C99)	
2	++ --	Prefix increment and decrement [note 1]	Right-to-left
	+-	Unary plus and minus	
	! ~	Logical NOT and bitwise NOT	
	(type)	Cast	
	*	Indirection (dereference)	
	&	Address-of	
	Sizeof	Size-of [note 2]	
	_Alignof	Alignment requirement(C11)	
3	* / %	Multiplication, division, and remainder	Left-to-right
4	+ -	Addition and subtraction	
5	<< >>	Bitwise left shift and right shift	

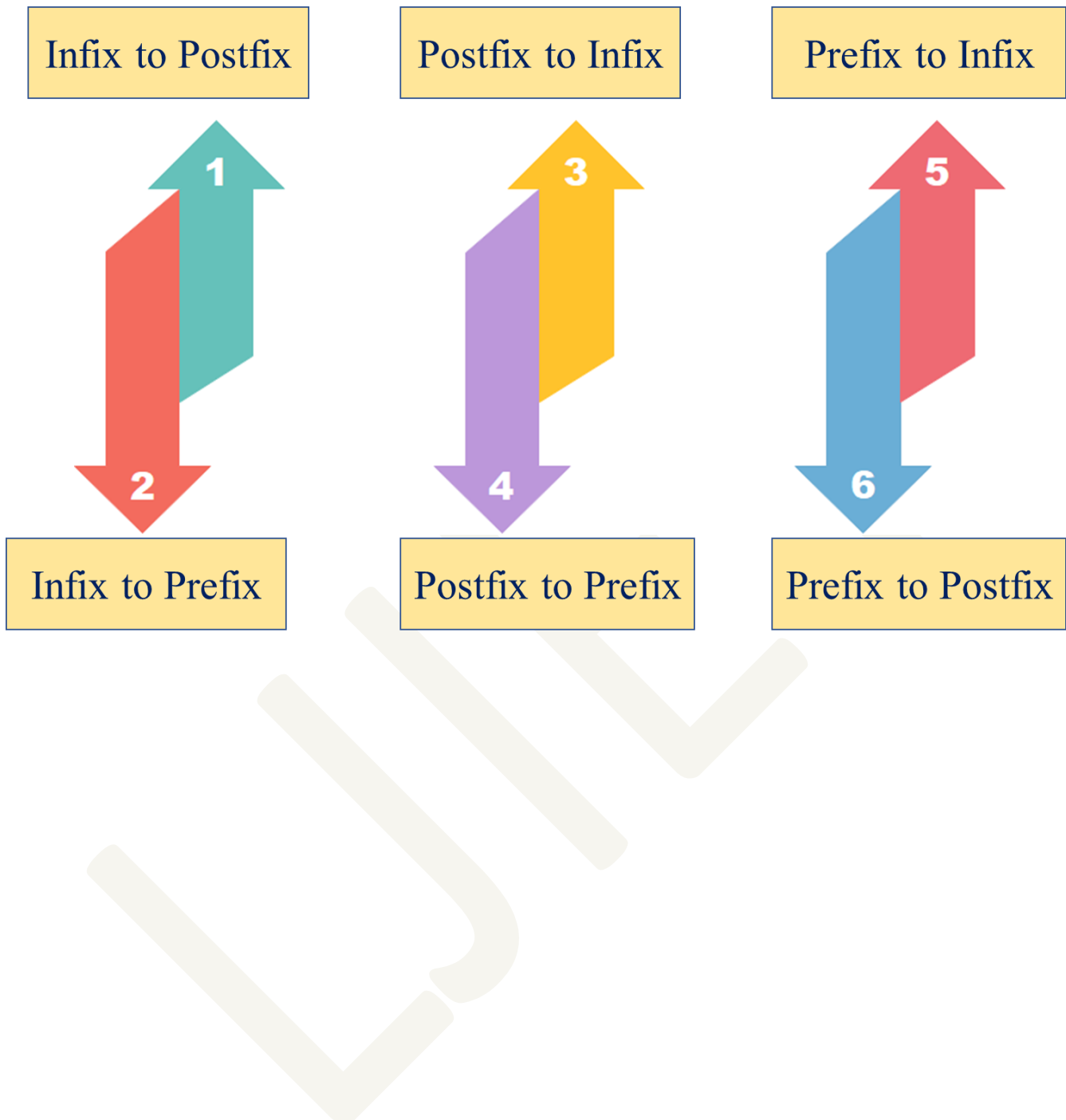
DATA STRUCTURES (017013292)

Semester – II

Chapter Name: STACK-2

6	< <=	For relational operators < and ≤ respectively	
	> >=	For relational operators > and ≥ respectively	
7	== !=	For relational = and ≠ respectively	
8	&	Bitwise AND	
9	^	Bitwise XOR (exclusive or)	
10		Bitwise OR (inclusive or)	
11	&&	Logical AND	
12		Logical OR	
13	?:	Ternary conditional[note 3]	Right-to-left
14	=	Simple assignment	
	+= -=	Assignment by sum and difference	
	*= /= %=	Assignment by product, quotient, and remainder	
	<<= >>=	Assignment by bitwise left shift and right shift	
	^= =	Assignment by bitwise AND, XOR, and OR	
15	,	Comma	Left-to-right

3.6 Expression Evaluation & Conversion



3.7 Infix to Postfix Conversion

Points to Consider:

The order of the numbers or operands remains unchanged. But the order of the operators gets changed in the conversion.

Stacks are used for converting an infix expression to a postfix expression. The stack that we use in the algorithm will change the order of operators from infix to Postfix.

Postfix expressions do not contain parentheses.

In order to write an algorithm we are dividing Infix expression in two parts

1) Infix expression with parenthesis

2) Infix expression without parenthesis

a. Infix to Postfix Conversion Without Parenthesis

Steps to convert Infix to postfix expression without parenthesis:

Step 1: Initialize stack contents to the special symbol #.

Step 2: Scan the left most symbol in the infix expression and denote it as the current input symbol.

Step 3: Repeat thru step 6 while the current input symbol is not #.

Step 4: Remove and output all stack symbols whose precedence values are greater than or equal to the precedence of the current input symbol.

Step 5: Push the current input symbol onto the stack.

Step 6: Scan the leftmost symbol in the infix expression and let it be the current input symbol.

Table of precedence:

Symbol	Precedence (Higher the int higher the precedence)	Rank
a,b,c....	3	1
*, /	2	-1
+, -	1	-1
#	0	-

b. ALGORITHM - Infix To Postfix Conversion Without Parenthesis

```
1. [Initialize the stack]
    TOP  $\leftarrow$  1
    S[TOP]  $\leftarrow$  '#'
2. [Initialize output string and rank count]
    POLISH  $\leftarrow$  ''
    RANK  $\leftarrow$  0
3. [Get first input symbol]
    NEXT  $\leftarrow$  NEXTCHAR(INFIX)
4. [Remove symbols with greater or equal precedence from stack]
Repeat while f(NEXT) <= f(S[TOP])
    TEMP  $\leftarrow$  POP(S, TOP) (this copies the stack content into TEMP)
    POLISH  $\leftarrow$  POLISH O TEMP (O is used for concatenation)
    RANK  $\leftarrow$  RANK + r(TEMP)
    If (RANK < 1)
    Then Write('Invalid')
    Exit
6. [Push current symbol onto the stack and obtain next input symbol]
    Call PUSH(S, TOP, NEXT)
    NEXT  $\leftarrow$  NEXTCHAR(INFIX)

7. [Remove remaining elements from stack]
Repeat while S[TOP]  $\neq$  '#'
    TEMP  $\leftarrow$  POP(S, TOP)
    POLISH  $\leftarrow$  POLISH O TEMP
    RANK  $\leftarrow$  RANK + r(TEMP)
    If RANK < 1
    then Write('Invalid')
    Exit

8. [is the expression valid?]
    If RANK = 1
    Then Write('Valid')
    Else Write('Invalid')

Exit
```

c. EXAMPLES – Infix Expression to Postfix Conversion Without Parenthesis**Rules:**

1. Priorities of Operators: ^ has the Top Priority then */ has the Next Priority then +- has the Least Priority.
2. No Two Operators of same priority can stay together in Stack Column.
3. Lowest Priority cannot be placed before the Highest Priority.

Question 1 – Convert the Following String from Infix to Postfix: $a + b * c - d / e * h$

Character scanned	Content of stack	Postfix	Rank
	#		
a	#a		
+	#+	a	1
b	#+b	a	1
*	#+*	ab	2
c	#+*c	ab	2
-	#-	abc*+	1
d	#-d	abc*+	1
/	#-/	abc*+d	2
e	#-/e	abc*+d	2
*	#-*	abc*+de/	2
h	#-*h	abc*+de/	2
#	#	abc*+de/h*-	1

Question 2 – Convert the Following String from Infix to Postfix: $a + b / c$

Character scanned	Content of stack	Postfix	Rank
	#		
a	#a		
+	#+	a	1
b	#+b	a	1
/	#+ /	ab	2
c	#+ / c	ab	2
#	#	abc/+	1

Question 3 – Convert the Following String from Infix to Postfix: $a + b / c * d$

Character scanned	Content of stack	Postfix	Rank
	#		
a	#a		
+	#+	a	1
b	#+b	a	1
/	#+ /	ab	2
c	#+ / c	ab	2
*	#+ *	abc/	2
d	#+ * d	abc/	2
#	#	abc/d*+	1

Question 4 – Convert the Following String from Infix to Postfix: $a - b - d * e / f + b * c$

Character scanned	Content of stack	Postfix	Rank
	#		
a	#a		
-	#-	a	1
b	#-b	a	1
-	#-	ab-	1
d	#-d	ab-	1
*	#-*	ab-d	2
e	#-*e	ab-d	2
/	#-/	ab-de*	2
f	#-/f	ab-de*	2
+	#+	ab-de*f/-	1
b	#+b	ab-de*f/-	1
*	#+*	ab-de*f/-b	2
c	#+*c	ab-de*f/-b	2
#	#	ab-de*f/-bc*+	1

Question 5 – Convert the Following String from Infix to Postfix: $a + b * c - d * e - f + g$

Character scanned	Content of stack	Postfix	Rank
	#		
a	#a		
+	#+	a	1
b	#+b	a	1
*	#+*	ab	2
c	#+*c	ab	2
-	#-	abc*+	1
d	#-d	abc*+	1
*	#-*	abc*+d	2
e	#-*e	abc*+d	2
-	#-	abc*+de*-	1
f	#-f	abc*+de*-	1
+	#+	abc*+de*-f-	1
g	#+g	abc*+de*-f-	1
#	#	abc*+de*-f-g+	1

Question 6 – Convert the Following String from Infix to Postfix: $e - x * c / b - e + f$

Character scanned	Content of stack	Postfix	Rank
	#		
e	#e		
-	#-	e	1
x	#-x	e	1
*	#-*	ex	2
c	#-*c	ex	2
/	#-/	exc*	2
b	#-/b	exc*	2
-	#-	exc*b/-	1
e	#-e	exc*b/-	1
+	#+	exc*b/-e-	1
f	#+f	exc*b/-e-	1
#	#	exc*b/-e-f+	1

Question 7 – Convert the Following String from Infix to Postfix: a^b^c/e

Character scanned	Content of stack	Postfix	Rank
	#		
a	#a		
^	#^	a	1
b	#^b	a	1
^	#^^	ab	2
c	#^^c	ab	2
/	#/	abc^^	1
e	#/e	abc^^	1
#	#	abc^^e/	1

Note: When dealing with Infix to Postfix problem that too without parenthesis, remember **two ^ can sit with each other in stack.**

Question 8 – Convert the Following String from Infix to Postfix: $K + L - M * N + O ^ P * W / U / V + T + Q$

Character scanned	Content of stack	Postfix	Rank
	#		
K	#K		
+	#+	K	
L	#+L	K	
-	#-	KL+	
M	#-M	KL+	
*	#-*	KL+M	
N	#-*N	KL+M	
+	#+	KL+MN*-	
O	#+O	KL+MN*-	
^	#+^	KL+MN*-O	
P	#+^P	KL+MN*-O	
*	#+*	KL+MN*-OP^	
W	#+*W	KL+MN*-OP^	
/	#+ /	KL+MN*-OP^W*	
U	#+ /U	KL+MN*-OP^W*	
/	#+ /	KL+MN*-OP^W*U/	
V	#+ /V	KL+MN*-OP^W*U/	
+	#+	KL+MN*-OP^W*U/V/+	
T	#+T	KL+MN*-OP^W*U/V/+	
+	#+	KL+MN*-OP^W*U/V/+T+	
Q	#+Q	KL+MN*-OP^W*U/V/+T+	
#	#	KL+MN*-OP^W*U/V/+T+Q+	

d. Infix to Postfix Conversion With Parenthesis

General Steps to convert into Infix to Postfix:

Step 1: Scan the infix expression from left to right.

Step 2: If the scanned character is an operand, output it.

Step 3: Else,

- If the precedence of the scanned operator is greater than the precedence of the operator in the stack (or the stack is empty or the stack contains a '('), push it.
- Else, Pop all the operators from the stack which are greater than or equal to in precedence than that of the scanned operator. After doing that Push the scanned operator to the stack. (If you encounter parenthesis while popping then stop there and push the scanned operator in the stack.)

Step 4: If the scanned character is an '(', push it to the stack.

Step 5: If the scanned character is an ')', pop the stack and output it until a '(' is encountered, and discard both the parenthesis.

Step 6: Repeat steps 2-6 until infix expression is scanned.

Step 7: Print the output.

Step 8: Pop and output from the stack until it is not empty.

Table of precedence:

Symbol	Input Precedence (Higher the int higher the precedence)	Stack Precedence	Rank
a, b, c....	7	8	1
↑	6	5	-1
*, /	3	4	-1
+, -	1	2	-1
(	9	0	-
)	0	-	-

e. ALGORITHM - Infix To Postfix Conversion With Parenthesis

```
1. [Initialize the stack]
    TOP  $\leftarrow$  1
    S[TOP]  $\leftarrow$  '('
2. [Initialize output string and rank count]
    POLISH  $\leftarrow$  ""
    RANK  $\leftarrow$  0
3. [Get first input symbol]
    NEXT  $\leftarrow$  NEXTCHAR(INFIX)
4. [Translate the infix expression]
    Repeat thru steps 7 while NEXT  $\neq$  ""
5. [Remove symbols with greater precedence from stack]
    If TOP < 1
    then Write('Invalid')
    Exit
Repeat while f(NEXT) < g(S[TOP])
    TEMP  $\leftarrow$  POP(S,TOP) (this copies the stack content into TEMP)
    POLISH  $\leftarrow$  POLISH O TEMP (O is used for concatenation)
    RANK  $\leftarrow$  RANK + r(TEMP)
    If (RANK < 1)
    Then Write('Invalid')
    Exit
6. [Are there matching parenthesis]
    If f(NEXT)  $\neq$  g(S[TOP])
    Then call PUSH(S, TOP, NEXT)
    Else POP(S,TOP)
7. [Get next input symbol]
    NEXT  $\leftarrow$  NEXTCHAR(INFIX)
8. [is the expression valid?]
    If TOP  $\neq$  0 or RANK  $\neq$  1
    Then Write('Invalid')
    Else Write('Valid')
Exit
```

Question 9 – Convert the Following String from Infix to Postfix: $(a + b \uparrow c \uparrow d) * (e + f / g)$

Character scanned	Content of stack	Postfix	Rank
	(		
(	((		
a	((a		
+	((+	a	1
b	((+b	a	1
$\uparrow$	((+ $\uparrow$	ab	2
c	((+ $\uparrow$ c	ab	2
$\uparrow$	((+ $\uparrow\uparrow$	abc	3
d	((+ $\uparrow\uparrow$ d	abc	3
)	(	abcd $\uparrow\uparrow+$	1
*	(*	abcd $\uparrow\uparrow+$	1
(	(*(	abcd $\uparrow\uparrow+$	1
e	(*(e	abcd $\uparrow\uparrow+$	1
+	(*(+	abcd $\uparrow\uparrow+e$	2
f	(*(+f	abcd $\uparrow\uparrow+e$	2
/	(*(+ /	abcd $\uparrow\uparrow+ef$	3
g	(*(+ /g	abcd $\uparrow\uparrow+ef$	3
)	(*	abcd $\uparrow\uparrow+efg/+$	2
)		abcd$\uparrow\uparrow+efg/+*$	1

Question 10 – Convert the Following String from Infix to Postfix: $(a+b) / c * d$

Character scanned	Content of stack	Postfix	Rank
	(		
(	((		
a	((a		
+	((+	a	1
b	((+b	a	1
)	(	ab+	1
/	(/	ab+	1
c	(/c	ab+	1
*	(*	ab+c/	1
d	(*d	ab+c/	1
)		ab+c/d*	1

Question 11 – Convert the Following String from Infix to Postfix: $(a+b) / (c-d) * e$

Character scanned	Content of stack	Postfix	Rank
	(		
(	((		
a	((a		
+	((+	a	1
b	((+b	a	1
)	(	ab+	1
/	(/	ab+	1
(	(/(	ab+	1
c	(/(c	ab+	1
-	(/(-	ab+c	2
d	(/(-d	ab+c	2
)	(	ab+cd-/	1
*	(*	ab+cd-/	1
e	(*e	ab+cd-/	1
)		ab+cd-/e*	1

Question 12 – Convert the Following String from Infix to Postfix: $a - (b+c) / (d/e)$

Character scanned	Content of stack	Postfix	Rank
	(		
a	(a		
-	(-	a	1
(	(-(	a	1
b	(-(b	a	1
+	(-(+	ab	2
c	(-(+c	ab	2
)	(-	abc+	2
/	(-/	abc+	2
(	(-/(	abc+	2
d	(-/(d	abc+	2
/	(-/(/	abc+d	3
e	(-/(/e	abc+d	3
)		abc+de//-	1
)		abc+de//-	1

Question 13 – Convert the Following String from Infix to Postfix: $(a - ((b - c) * d) + f)$

Character scanned	Content of stack	Postfix	Rank
	(		
(	((		
a	((a		
-	((-	a	1
(	((-(	a	1
(	(((-(	a	1
b	(((-(b	a	1
-	(((-(-	ab	2
c	(((-(c	ab	2
)	((-(	abc-	2
*	(((-*	abc-	2
d	(((-(*d	abc-	2
)	(((-	abc-d*	2
+	(((+	abc-d*-	1
f	(((+f	abc-d*-	1
)		abc-d*-f+	1
)		abc-d*-f+	1

3.8 Postfix to Infix Conversion Using Stack

Postfix Expression can easily solve by machine(computers) but for human postfix operation is difficult to evaluate. In the back-end side(machine) Postfix expression is necessary and in the front-end side (user) Infix expression is necessary.

So let's Study how to convert Postfix expression into infix expression.

Steps to convert POSTFIX TO INFIX Expression:

Step 1: Read the symbol from the input based on the input symbol go to step 2 or 3

Step 2: If the symbol is an operand, then push it onto the Stack

Step 3: If the symbol is an operator, then pop two operands from the Stack

Step 4: Create a string by concatenating the two operands and the operator between them.

string = (operand1 + operator + operand2)

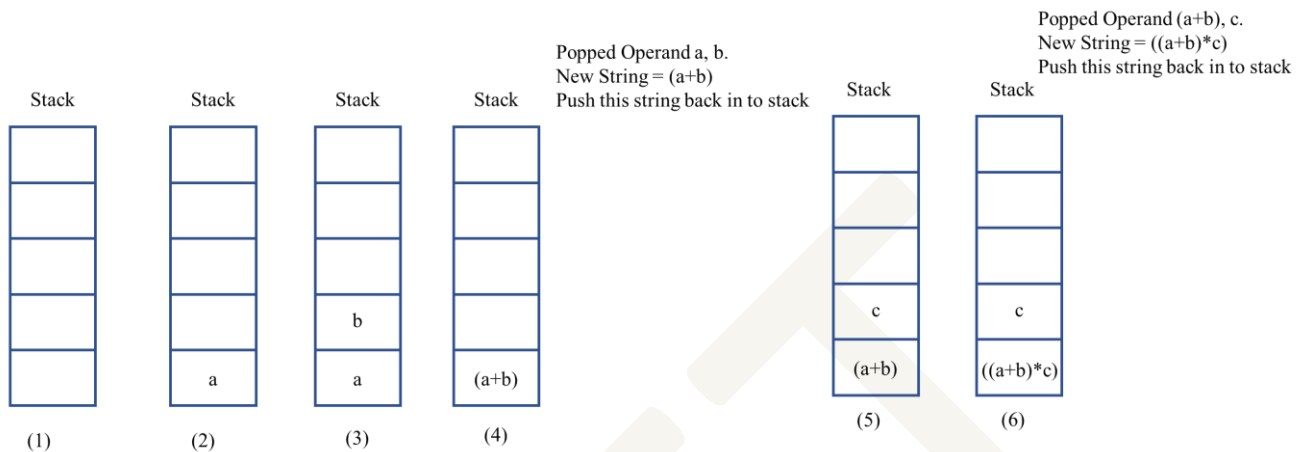
And push the resultant string back to Stack

Step 5: Repeat the above steps until the end of Prefix expression.

Step 6: At the end stack will have only 1 string i.e resultant string

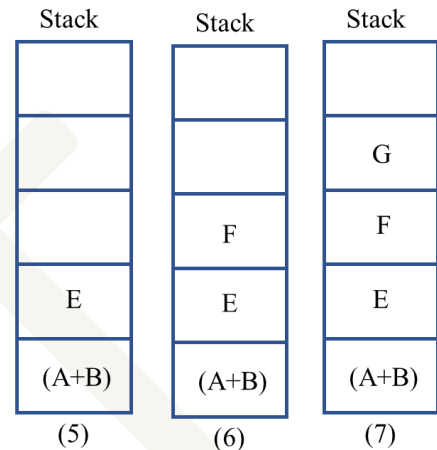
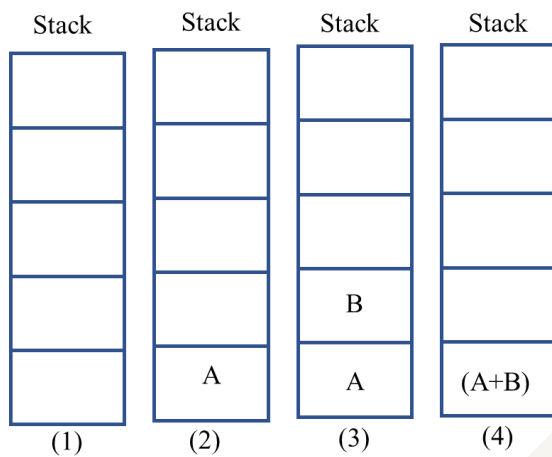
a. EXAMPLES – Postfix Expression to Infix Conversion Using Stacks

Question 1 – Convert the Following String from Postfix to Infix using Stack: $ab+c^*$

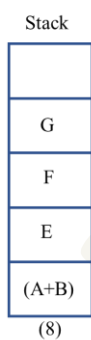


Question 2 – Convert the Following String from Postfix to Infix using Stack: $AB+EFG/+*$

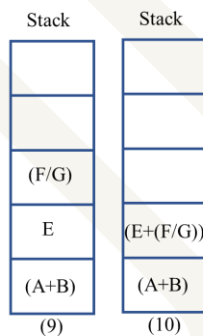
Popped Operand A, B.
New String = $(A+B)$
Push this string back in to stack



Popped Operand G & F.
New String = (F/G)
Push this string back in to stack



Popped Operand (F/G) & E.
New String = $(E+(F/G))$
Push this string back in to stack



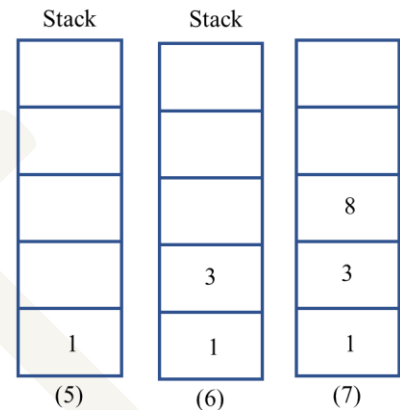
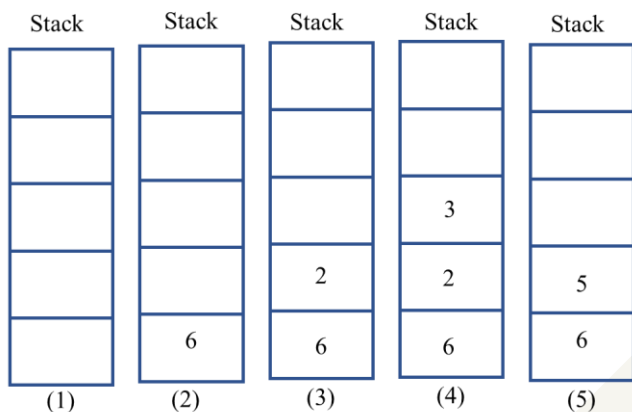
Popped Operand $(E+(F/G))$ & $(A+B)$.
New String = $((A+B)*(E+(F/G)))$
Push this string back in to stack



Question 3 – Evaluate: $623+-382/+*2^3+$

Popped Operand 3 & 2.
New String = $(2+3) = 5$
Push this string back in to stack

Popped Operand 5 & 6.
New String = $(6-5) = 1$
Push this string back in to stack

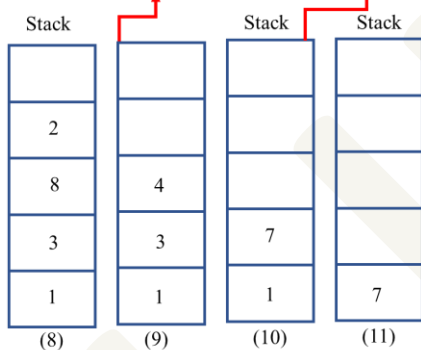


Popped Operand 2 & 8.
New String = $(8/2) = 4$
Push this string back in to stack

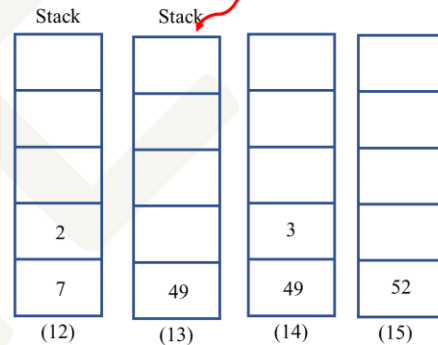
Popped Operand 4 & 3.
New String = $(3+4) = 7$
Push this string back in to stack

Popped Operand 2 & 7.
New String = $(7^2) = 49$
Push this string back in to stack

Popped Operand 3 & 49.
New String = $(49+3) = 52$
Push this string back in to stack



Popped Operand 7 & 1.
New String = $(7*1) = 7$
Push this string back in to stack



Note – When you first pop a operand from the stack that is operand2 and then the next operand popped is operand 1. Hence the Infix will become operand1 <operator> operand2 in our case $2+3 = 5$

3.9 Infix to Prefix Conversion

- ▶ While we use infix expressions in our day to day lives. Computers have trouble understanding this format because they need to keep in mind rules of operator precedence and also brackets.
- ▶ Prefix and Postfix expressions are easier for a computer to understand and evaluate.
- ▶ Given two operands a and b and an operator +
- ▶ When the operator is placed before the operands i.e +ab the expression in prefix notation.
- ▶ Given any infix expression, we can obtain the equivalent prefix format.

Steps to convert Infix to prefix expression:

Step 1: Reverse the infix expression i.e $A+B*C$ will become $C*B+A$. Note while reversing each '(' will become ')' and each ')' becomes '('.

Step 2: Obtain the “nearly” postfix expression of the modified expression i.e $CB*A+$.

Step 3: Reverse the postfix expression. Hence in our example prefix is $+A*BC$.

Note that for Step 2, we don't use the postfix algorithm as it is. There is a minor change in the algorithm. As per infix to postfix algorithm, we have to pop all the operators from the stack which are greater than or equal to in precedence than that of the scanned operator. But here, we have to pop all the operators from the stack which are greater in precedence than that of the scanned operator. Only in the case of “^” operator, we pop operators from the stack which are greater than or equal to in precedence.

a. Examples - Infix to Prefix Conversion Without Parenthesis**Question 1** – Convert the Following String from Infix to Prefix: $a+b*e+f/g$ **Step 1:** Reverse the given String: $g/f+e*b+a$ **Step 2:** Find Nearly PostFix

Character scanned	Content of stack	Postfix	Rank
	#		
g	#g		
/	#/	g	1
f	#/f	g	1
+	#+	gf/	1
e	#+e	gf/	1
*	#+*	gf/e	2
b	#+*b	gf/e	2
+	#++	gf/eb*	2
a	#++a	gf/eb*	2
#	#	gf/eb*a++	1

Step 3: Reverse the Final Answer, that is: **++a*be/fg**

Question 2 – Convert the Following String from Infix to Prefix: a^b^c/d^*e-f

Step 1: Reverse the given String: **$f-e^*d/c^b^a$**

Step 2: Find Nearly PostFix

Character scanned	Content of stack	Postfix	Rank
	#		
f	#f		
-	#-	f	1
e	#-e	f	1
*	#-*	fe	2
d	#-*d	fe	2
/	#-*/	fed	3
c	#-*/c	fed	3
^	#-*/^	fedc	4
b	#-*/^b	fedc	4
^	#-*/^	Fedcb^	5
a	#-*/^a	Fedcb^	5
	#	fedcb^a^/*-	1

Step 3: Reverse the Final Answer, that is: **$-*/^a^b^c^d^e^f$**

Question 3 – Convert the Following String from Infix to Prefix: $P \$ Q \$ R / S * T * U - V + W$

Step 1: Reverse the given String: $W + V - U * T * S / R \$ Q \$ P$

Step 2: Find Nearly PostFix

Character scanned	Content of stack	Postfix	Rank
	#		
W	#W		
+	#+	W	1
V	#+V	W	1
-	#+-	WV	2
U	#+-U	WV	2
*	#+-*	WVU	3
T	#+-*T	WVU	3
*	#+-**	WVUT	4
S	#+-**S	WVUT	4
/	#+-**/	WVUTS	5
R	#+-**/R	WVUTS	5
\$	#+-**/\$	WVUTSR	6
Q	#+-**/\$Q	WVUTSR	6
\$	#+-**/\$	WVUTSRQ\$	6
P	#+-**/\$P	WVUTSRQ\$	6
	#	WVUTSRQ\$P\$/**-+	1

Step 3: Reverse the Final Answer, that is: $+ - ** / \$ P \$ Q R S T U V W$

b. Examples - Infix to Prefix Conversion With Parenthesis**Question 4** – Convert the Following String from Infix to Prefix: $(a+b)/d$ **Step 1:** Reverse the given String: **d/)b+a(****Step 2:** Find Nearly PostFix

Character scanned	Content of stack	Postfix	Rank
	)		
d	)d		
/	)d/	d	1
)	)d/)	d	1
b	)d/)b	d	1
+	)d/)+	db	2
a	)d/)+a	db	2
(	)d/)	dba+	2
(		dba+/ d	1

Step 3: Reverse the Final Answer, that is: **/+abd**

Question 5 – Convert the Following String from Infix to Prefix: $(A*B) + (C+D)$

Step 1: Reverse the given String: $)D+C(+)B*A($

Step 2: Find Nearly PostFix

Character scanned	Content of stack	Postfix	Rank
	)		
)	))		
D	))D		
+	))+	D	1
C	))+C	D	1
(	)	DC+	1
+	)+	DC+	1
)	)+)	DC+	1
B	)+)B	DC+	1
*	)+)*	DC+B	2
A	)+)*A	DC+B	2
(	)+	DC+BA*	2
(		DC+BA*+	1

Step 3: Reverse the Final Answer, that is: **+*AB+CD**

Question 6 – Convert the Following String from Infix to Prefix: $K+L-M*N+(O^P)*W/U/V+T+Q$

Step 1: Reverse the given String: $Q+T+V/U/W*)P^O(+N*M-L+K$

Step 2: Find Nearly PostFix

Character scanned	Content of stack	Postfix	Rank
	)		
Q	)Q		
+	)+	Q	1
T	)T	Q	1
+	)++	QT	2
V	)++V	QT	2
/	)++/	QTV	3
U	)++/U	QTV	3
/	)++//	QTVU	4
W	)++//W	QTVU	4
*	)++//*	QTVUW	5
)	)++//*)	QTVUW	5
P	)++//*)P	QTVUW	5
^	)++//*)^	QTVUWP	6
O	)++//*)^O	QTVUWP	6
(	)++//*	QTVUWP O^	6
+	)+++	QTVUWP O^*//	3
N	)+++N	QTVUWP O^*//	3
*	)+++*	QTVUWP O^*//N	4
M	)+++*M	QTVUWP O^*//N	4
-	)+++ -	QTVUWP O^*//NM*	4
L	)+++ -L	QTVUWP O^*//NM*	4
+	)+++ -+	QTVUWP O^*//NM*L	5
K	)+++ -+K	QTVUWP O^*//NM*L	5
(		QTVUWP O^*//NM*LK+-+++	1

Step 3: Reverse the Final Answer, that is: **+++ -+KL*MN//*^OPWUVTQ**

Question 7 – Convert the Following String from Infix to Prefix: $A-B+(M\$N)*(O+P)-Q/R^{\wedge}S^*T+Z$

Step 1: Reverse the given String: $Z+T^*S^{\wedge}R/Q-)P+O(*)N\$M(+B-A$

Step 2: Find Nearly PostFix

Character scanned	Content of stack	Postfix	Rank
	)		
Z	)Z		
+	)+	Z	
T	)T	Z	
*	)T*	ZT	
S	)T*S	ZT	
^	)T*S^	ZTS	
R	)T*S^R	ZTS	
/	)T*S^R/	ZTSR^	
Q	)T*S^R/Q	ZTSR^	
-	)T*S^R/Q-	ZTSR^Q/*	
)	)T*S^R/Q-)	ZTSR^Q/*	
P	)T*S^R/Q-)P	ZTSR^Q/*	
+	)T*S^R/Q-)P+	ZTSR^Q/*P	
O	)T*S^R/Q-)P+O	ZTSR^Q/*P	
(	)T*S^R/Q-)P+O(	ZTSR^Q/*PO+	
*	)T*S^R/Q-)P+O(*	ZTSR^Q/*PO+	
)	)T*S^R/Q-)P+O(*)	ZTSR^Q/*PO+	
N	)T*S^R/Q-)P+O(*)N	ZTSR^Q/*PO+	
\$	)T*S^R/Q-)P+O(*)N\$	ZTSR^Q/*PO+N	
M	)T*S^R/Q-)P+O(*)N\$M	ZTSR^Q/*PO+N	
(	)T*S^R/Q-)P+O(*)N\$M(	ZTSR^Q/*PO+NM\$	
+	)T*S^R/Q-)P+O(*)N\$M(+	ZTSR^Q/*PO+NM\$*	
B	)T*S^R/Q-)P+O(*)N\$M(+B	ZTSR^Q/*PO+NM\$*	
-	)T*S^R/Q-)P+O(*)N\$M(+B-	ZTSR^Q/*PO+NM\$*B	
A	)T*S^R/Q-)P+O(*)N\$M(+B-A	ZTSR^Q/*PO+NM\$*B	
(	)T*S^R/Q-)P+O(*)N\$M(+B-A(	ZTSR^Q/*PO+NM\$*BA-+-+	

Step 3: Reverse the Final Answer, that is: $+--AB^*\$MN+OP^*/Q^{\wedge}RSTZ$

3.10 Prefix to Infix Conversion Using Stack

Steps to convert **PREFIX TO INFIX** expression:

Step 1: Read the Prefix expression in reverse order (from right to left)

Step 2: If the symbol is an operand, then push it onto the Stack

Step 3: If the symbol is an operator, then pop two operands from the Stack

Step 4: Create a string by concatenating the two operands and the operator between them.

$\text{string} = \text{operand1} + \text{operator} + \text{operand2}$

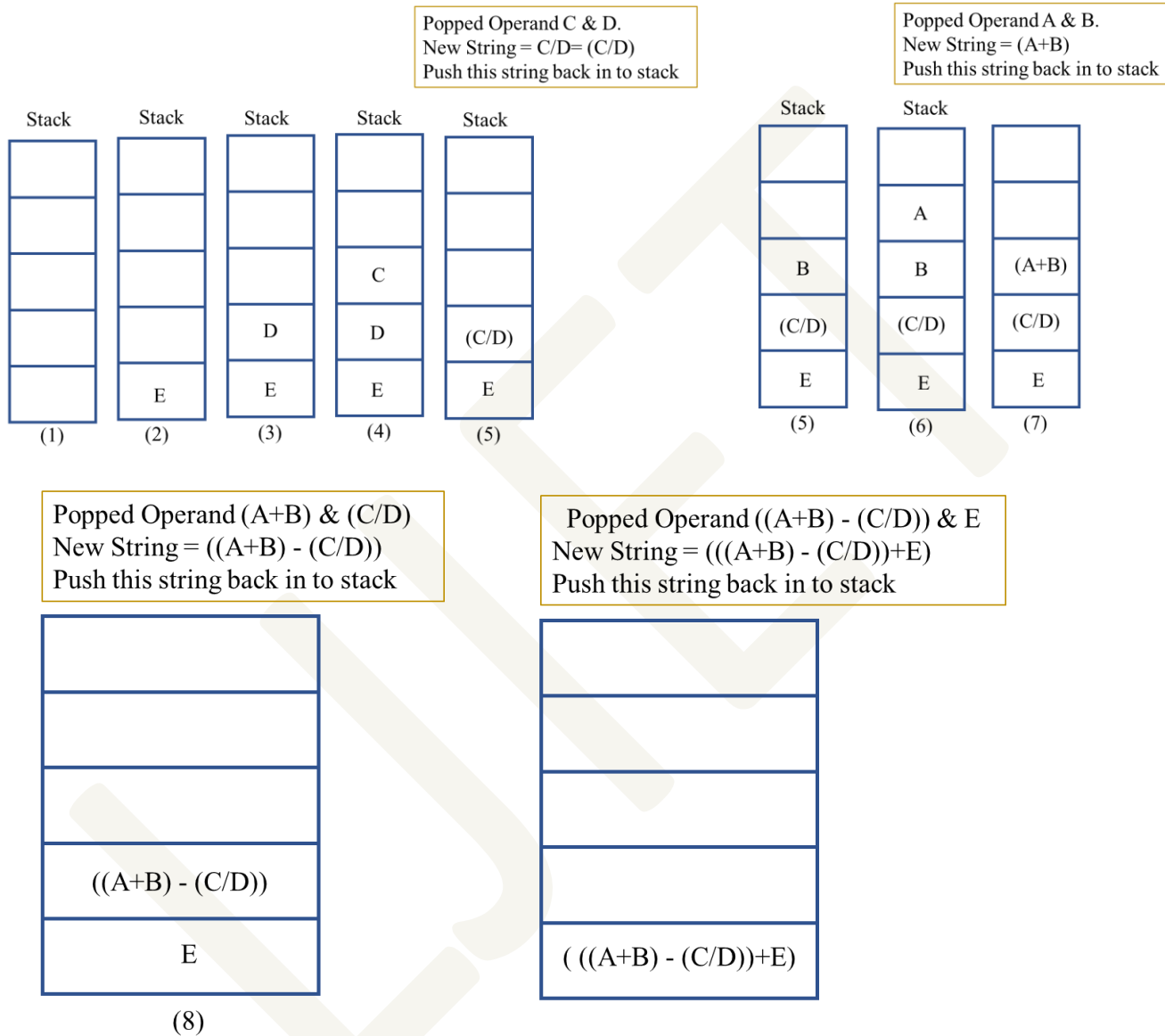
And push the resultant string back to Stack

Step 5: Repeat the above steps until end of Prefix expression.

a. Examples – Prefix to Infix Conversion Using Stack

Question 1 – Convert the Following String from Prefix to Infix using Stack: $+ - + AB / CDE$

Reverse the given equation – $EDC / BA + - +$

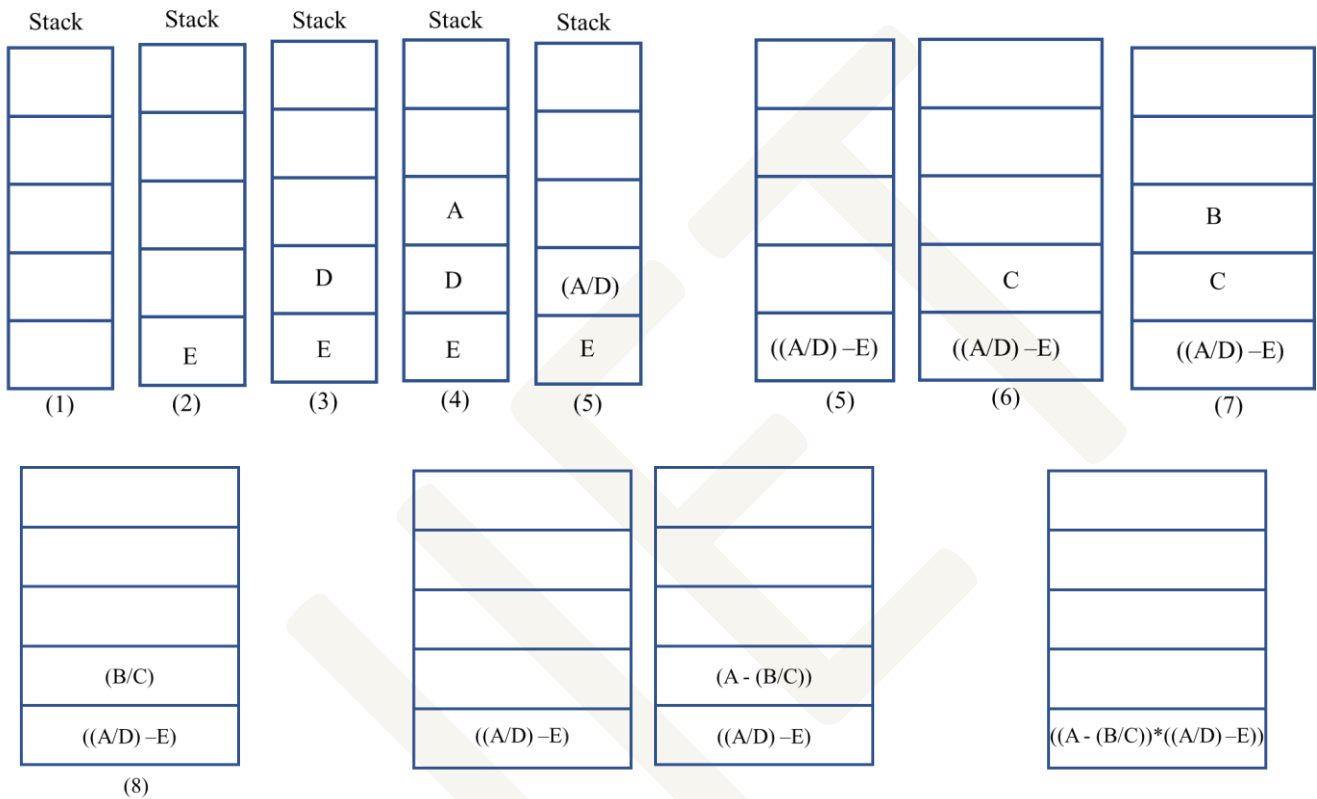


Note – When you first pop a operand from the stack that is operand1 and then the next operand popped is operand 2. Hence the Infix will become operand1 <operator> operand2 in our case $C / D = (C/D)$

Question 2 – Convert the Following String from Prefix to Infix using Stack: $*-A/BC-/ADE$

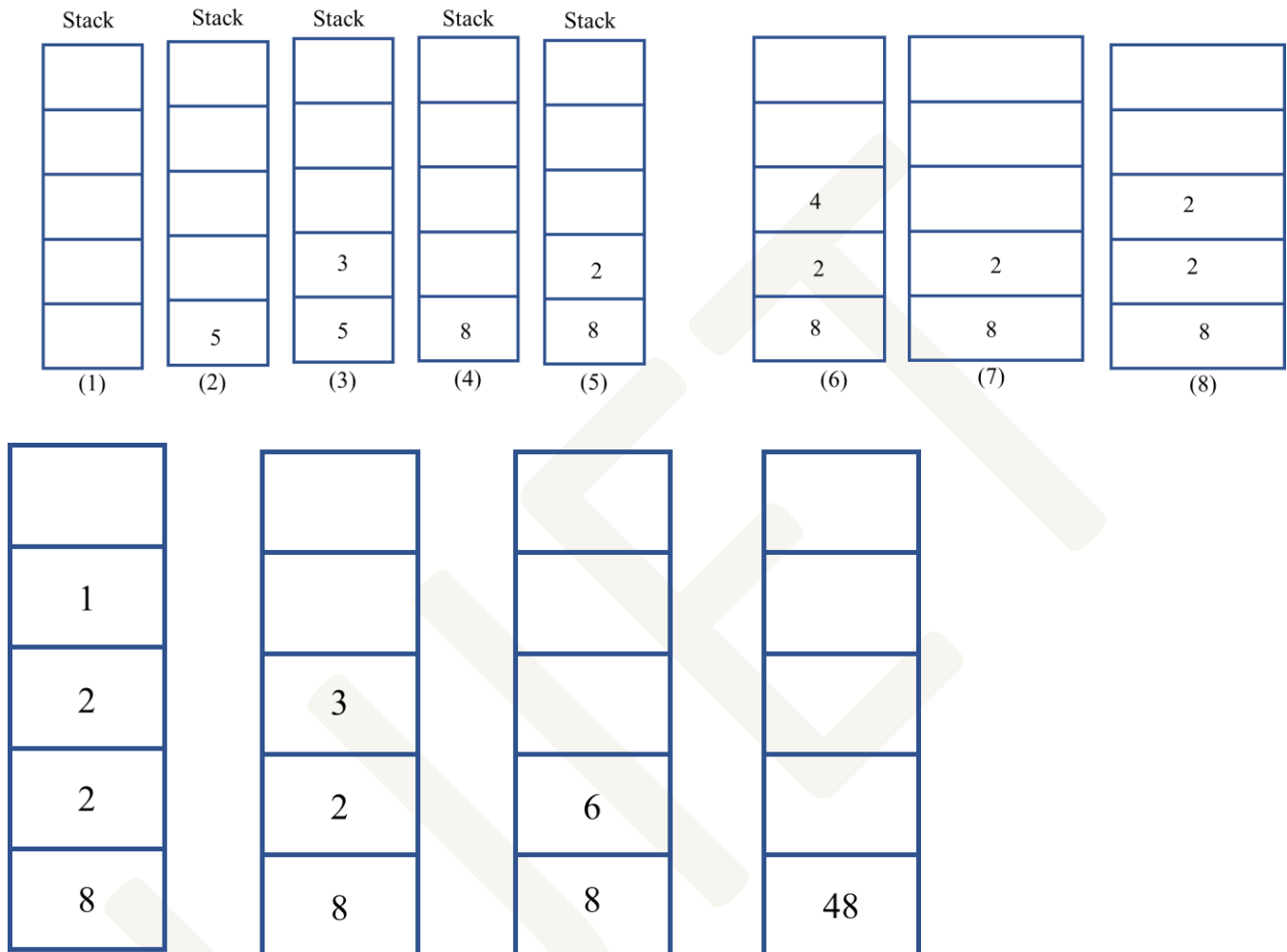
Reverse the given equation – $EDA/-CB/A-*$

Popped Operand A & D
New String = $A/D = (A/D)$
Push this string back in to stack



Question 3 – What would be the solution to the given prefix notation? * * + 1 2 / 4 2 + 3 5

Reverse the given equation – 5 3 + 2 4 / 2 1 + * *



3.11 Prefix to Postfix Conversion Using Stack

Steps to convert **PREFIX TO POSTFIX** expression:

Step 1: Read the Prefix expression in reverse order (from right to left)

Step 2: If the symbol is an operand, then push it onto the Stack

Step 3: If the symbol is an operator, then pop two operands from the Stack

Step 4: Create a string by concatenating the two operands and the operator after them.

string = operand1 + operand2 + operator

And push the resultant string back to Stack

Step 5: Repeat the above steps until end of Prefix expression.

a. Examples - Prefix to Postfix Conversion Using Stack

Question 1 – Convert the Following String from Prefix to Postfix using Stack: $++ab/cd*gh$

